Inference Without a Formula

October 23, 2026

Where we left off

- Unit 5 ended with a tree finding a threshold, and a regression turning that threshold into a number with an interval on it.
- The regression did the reporting because the tree could not. It has no coefficients and no standard errors, so there was nothing to report.
- That was a workaround. This unit is about getting those answers out of the tree itself.

What this unit gives you

Things to know

- what a variable importance score measures, and what it does not
- why the default importance in most software flatters some columns
- what permutation importance does instead
- how resampling your own data produces an interval
- how to put an interval on a quantity with no formula
- what resampling cannot repair

Mistakes to stop making

- reading an importance ranking as a ranking of effects
- trusting the default importance on a column with many distinct values
- quoting a single number for anything resampling could have given an interval for
- resampling before splitting the data, so the same row lands on both sides
- treating a bootstrap interval as a repair for a biased sample

What a model without a likelihood cannot hand you

Unit 2 drew the line between a model that states a distribution for the outcome and one that does not. Everything on the right of that line costs something:

From a regression	**From a forest**
a coefficient, in units	nothing directly comparable
a standard error	nothing
a confidence interval	nothing
a p-value	nothing
AIC	nothing, there is no likelihood

None of those gaps is permanent. Resampling rebuilds the middle three, and an importance score covers part of the first. What they cost is compute time rather than a new assumption.

The data for this unit

`rent.csv`, the 4,743 rental listings from the Unit 5 homework. Same data, a different question: not how much a square foot is worth, but which columns the model leans on and how sure we are of any of it.

Column	What it is
`Rent`	monthly rent in rupees, the outcome
`Size`	floor area in square feet
`BHK`, `Bathroom`	bedrooms, bathrooms
`City`	six cities
`FurnishingStatus`	furnished, semi, unfurnished
`id_number`	**added here**: a random six-digit number, meaning nothing

That last column is the test. A column that cannot possibly matter tells you which importance methods can be believed.

What an importance score is

- A forest has no coefficient for `Size`, so there is no sentence of the form "a square foot is worth this much".
- What it can report is how much the column contributed to the fitting. That is a **variable importance** score.
- It is one number per column, always positive or near zero, with no units and no direction. It cannot tell you whether rent goes up or down with size.

Trap: the word does a lot of work

Important is not the same as large, and not the same as causal. An importance score ranks columns by how much this particular fitted model used them.

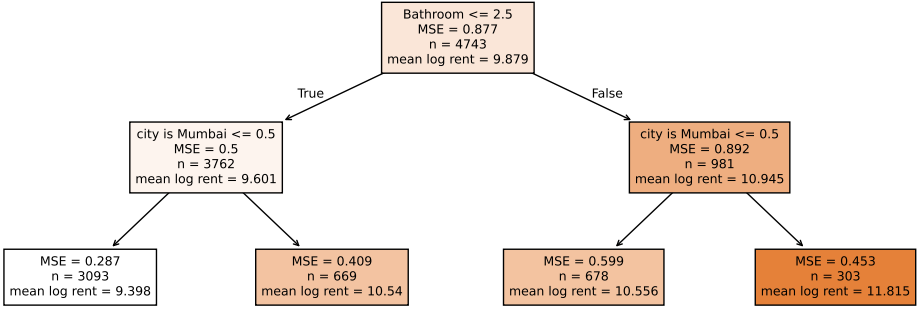
The default: how much splitting a column did

- Every split in every tree improves the fit by some amount. Add up those improvements for each column and you have **impurity importance**, which is what `.feature_importances_` returns.
- It is free, because the forest already did the arithmetic while it was being built.
- It also has a known flaw: a column with many distinct values gets many candidate cut points, so it wins splits by having more chances, not by carrying more signal.

The formula, and a tree to try it on

Every split cuts the spread inside a node. Weight that cut by how many rows reached the node:

$$\text{importance}(j) \;=\; \frac{1}{N} \sum_{\text{nodes splitting on } j} \Big(n_t \,\text{MSE}_t \;-\; n_L \,\text{MSE}_L \;-\; n_R \,\text{MSE}_R \Big)$$



The same tree, by hand

Split on	Arithmetic	Decrease
bathrooms, at the root	$(4743)(0.877) - (3762)(0.500) - (981)(0.892)$, over 4743	0.296
Mumbai, left branch	$(3762)(0.500) - (3093)(0.287) - (669)(0.409)$, over 4743	0.151
Mumbai, right branch	$(981)(0.892) - (678)(0.599) - (303)(0.453)$, over 4743	0.070

Bathrooms gets 0.296. Mumbai splits twice and collects $0.151 + 0.070 = 0.221$. Over the total 0.517 that is **0.573** and **0.427**, which is what `.feature_importances_` prints.

Trap: where the bias enters

Nothing in that sum asks whether a column deserved its splits. A column with a distinct value in every row is offered a cut point between every pair, so it wins more nodes.

How to catch it

- Add a column of random six-digit numbers to the listings. It has nothing to do with rent. We know this, because we generated it.
- Fit the forest on everything, including the random column.
- If an importance method is trustworthy, that column should come out at the bottom.

In practice: a sanity check you can always run

A column you know to be noise, added on purpose, is the cheapest audit of any importance method, any feature selection routine, and any claim that a model found something.

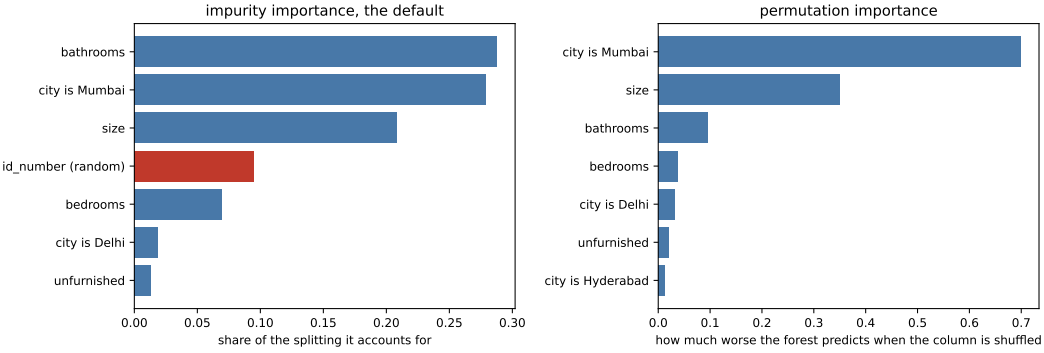
Permutation importance, and why it is harder to fool

1. Fit the model. Measure how well it predicts on rows it was not fitted on.
2. Take one column and shuffle it, breaking any relationship it had with the outcome while leaving everything else alone.
3. Measure again. The drop in performance is that column's importance.
4. Put the column back and do the next one.

Principle: why the random column scores zero

Shuffling a column that was doing nothing changes nothing, so the score lands at zero and can even come out slightly negative. Shuffling `Size` costs the forest 0.350.

Two methods, two different answers



On the left, the random column places **fourth of eleven**, above bedrooms. On the right it does not reach the top seven. The two methods also disagree about the real columns: the left ranks bathrooms first, the right ranks Mumbai first by a wide margin.

The same five columns, ranked both ways

Column	impurity		permutation	
bathrooms	0.288	(1st)	0.096	(3rd)
city is Mumbai	0.279	(2nd)	0.699	(1st)
size	0.208	(3rd)	0.350	(2nd)
`id_number`, random	**0.095**	**(4th)**	-0.002	**(11th)**
bedrooms	0.070	(5th)	0.038	(4th)

Trap: the default is the one most people report

Nine percent of the credit went to a column of random digits, and the method that did it is the one that runs by default and costs nothing.

Which one to run

Situation	What to do
reporting to anyone	permutation importance, on held-out rows
an id, timestamp or postcode is in the data	permutation importance, and do not quote the default at all
a quick look while building	the default is free and fine, just keep it out of the write-up
columns that are near-copies	neither is reliable, they split the credit. Drop one first
you need a direction or a size	neither. Fit a regression, or read a partial dependence curve

In practice: one line of code

```
permutation_importance(model, X_test, y_test, n_repeats=10)
```

Quick check: reading an importance table

Quick check: thirty seconds

A forest on house prices reports permutation importance: square footage 0.41, neighborhood 0.22, roof material 0.01. A colleague writes: "square footage matters about twice as much as neighborhood, and roof material raises price by 0.01."

Two things are wrong. What are they?

Quick check: reading an importance table

Quick check: thirty seconds

A forest on house prices reports permutation importance: square footage 0.41, neighborhood 0.22, roof material 0.01. A colleague writes: "square footage matters about twice as much as neighborhood, and roof material raises price by 0.01."

Two things are wrong. What are they?

Answer

The ratio. Importance is not on a scale where twice the number means twice the effect. It says the forest relied on square footage more, not by how much more. **The units.** 0.01 is a drop in predictive performance, not dollars, and it has no direction. Nothing here says roof material raises anything.

The question an importance score cannot answer

- The random column scored -0.002 and bedrooms scored 0.038. Is that a real difference, or would another sample of listings have put them the other way round?
- For a regression coefficient there is a formula for that, and it gives the standard error.
- For an importance score there is no formula. There is also no formula for a median, for a ratio of two medians, or for most things anyone actually wants.

Principle: the gap

The question is how much an answer would move if you had collected a different sample. Without a formula, the only way to find out is to watch it move.

Resampling your own data

- You cannot go back and collect 4,743 more listings. What you have is the one sample.
- So treat that sample as a stand-in for the population, and draw new samples from it: **the same size, with replacement**, so some rows appear twice and some not at all.
- Compute your answer on each new sample. The spread of those answers estimates the spread you would have seen across real samples.

Trap: with replacement is not optional

Drawing 4,743 rows without replacement returns the original data in a different order, every time, and the spread comes out as zero.

The recipe

1. Draw a resample of the same size, with replacement.
2. Compute the quantity you care about on it, whatever that quantity is.
3. Keep the number.
4. Repeat a few hundred to a few thousand times.
5. For a 95% interval, take the 2.5th and 97.5th percentiles of the numbers you kept.

In practice: what step 2 is allowed to be

Anything you can compute. A mean, a median, a coefficient, a ratio, an importance score, the output of a forest, the gap between two groups. The recipe does not care.

The settings, and what they are worth arguing about

Choice	What to use
how many resamples	a couple of hundred while you are working, one or two thousand for anything you report
resample size	the same as the original. Not a round number, not a fraction of it
which percentiles	2.5 and 97.5 for a 95% interval. 5 and 95 if you want 90%
a random seed	set one, so the interval you quote is the interval you can reproduce

In practice: how much the count matters

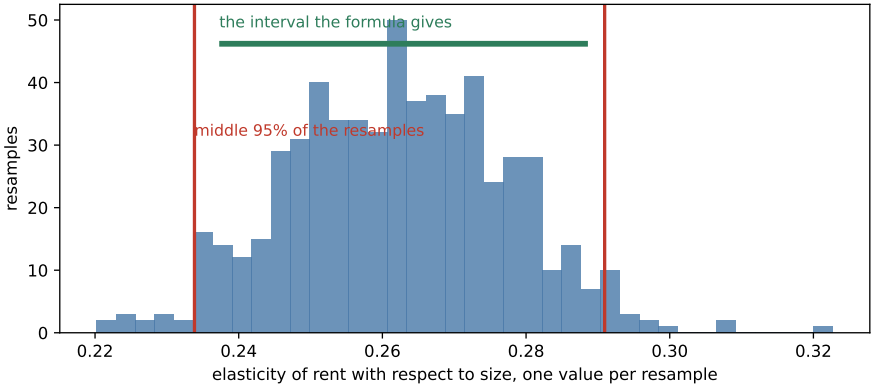
On the Mumbai and Delhi gap, 50 resamples moved the reported endpoints by about Rs 1,000 from run to run. At 200 they stopped moving. Past a thousand you are buying nothing.

In code

```
boot = []
for i in range(2000):
  sample = rents.sample(len(rents), replace=True)
  boot.append( whatever_you_want(sample) )
np.percentile(boot, [2.5, 97.5])
```

Five lines, and the third one is the whole method. Everything specific to your problem goes inside `whatever_you_want`, and nothing else changes.

Does it actually work?



The elasticity of rent with respect to size, computed 600 times on 600 resamples. The formula gives $[0.237, 0.289]$ and the middle 95% of the resamples is $[0.234, 0.291]$. Where a formula exists, resampling reproduces it.

Quick check: what the resamples are for

Quick check: thirty seconds

A student runs 2,000 resamples, takes the average of the 2,000 answers, and reports that as the estimate instead of the one computed on the original data.

Is that a reasonable thing to do, and what were the 2,000 numbers for?

Quick check: what the resamples are for

Quick check: thirty seconds

A student runs 2,000 resamples, takes the average of the 2,000 answers, and reports that as the estimate instead of the one computed on the original data.

Is that a reasonable thing to do, and what were the 2,000 numbers for?

Answer

The average of the resamples is close to the original estimate and is not an improvement on it; resampling adds no information that was not already in the sample. The 2,000 numbers are there for their *spread*, which is what you had no other way of getting. Report the estimate from the data and the interval from the resamples.

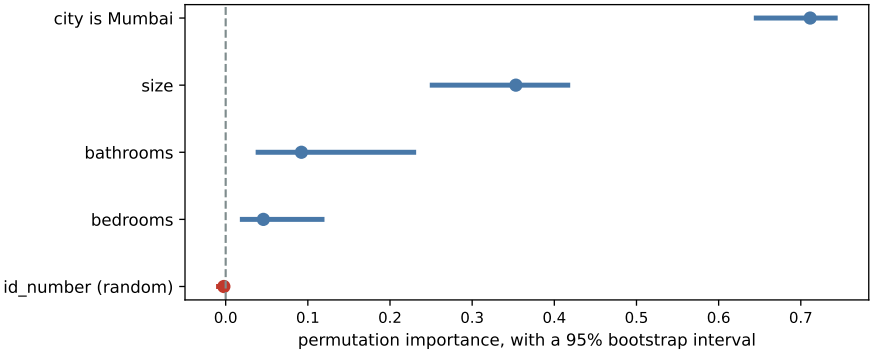
A gap between two medians

- Median rent in Mumbai is Rs 52,000. In Delhi it is Rs 17,000. The gap is **Rs 35,000**.
- There is no standard error formula for a difference of medians in any textbook you are likely to open.
- Resample the Mumbai listings and the Delhi listings separately, take the gap each time, and read off the percentiles: **Rs 32,000 to Rs 39,000**.

In practice: the reporting sentence

"Mumbai's median rent runs about Rs 35,000 above Delhi's, and the data pins that gap down to somewhere between Rs 32,000 and Rs 39,000."

An interval on an importance score



The same permutation importances, each with a 95% bootstrap interval. Four columns sit clearly away from zero. The random one straddles it, which is the answer to the question the ranking alone could not settle.

The one way to get this wrong

- The importance was measured on held-out rows, so the resampling has to leave that split alone: resample the **training rows only** and score against a test set that was never resampled.
- Resample first and split afterwards, and the same listing lands in both halves. The forest has seen the test rows, shuffling any column then hurts it, and **every** column scores as important, the random one included.

Trap: it fails quietly and in the flattering direction

Nothing errors. The intervals come back narrow and entirely above zero, and the random column looks like a finding.

What to resample, case by case

What you are estimating	What one resample is
a mean, a median, a percentile	the rows
a gap between two groups	each group separately, each keeping its own size
a regression coefficient	the rows, then refit the whole model
anything scored on held-out rows	the training rows only, test set untouched
whether model A beats model B	the rows, then refit *both* and keep the difference
rows that come in clusters	the clusters, not the rows inside them

Principle: the rule underneath the table

Resample whatever you would have had to collect again. If a second sample would have meant 4,743 new listings, resample listings. If it would have meant 60 new buildings, resample buildings.

Your turn #1

Your turn: what would you resample?

For each one, say what a single resample consists of and what you compute on it:

1. The 90th percentile of repair times, with an interval.
2. The difference in median salary between two departments.
3. Whether a forest's accuracy is really better than a regression's on the same data.

Your turn #1: answer

- **(1)** Resample the repair times with replacement, take the 90th percentile of each resample, read the percentiles of those.
- **(2)** Resample each department separately, take the gap in medians each time. Keep the two groups separate so each keeps its own sample size.
- **(3)** Resample the rows, refit *both* models on each resample, and keep the difference in held-out error. If the interval for the difference covers zero, the data does not establish that one is better. Refitting both on the same resample is the part people skip.

What resampling cannot repair

- **A biased sample.** If the listings over-represent agents, every resample over-represents agents too. The interval will be tight around the wrong number.
- **Rows that are not independent.** Several listings from one building are not several pieces of evidence. Resample rows and the interval comes out too narrow.
- **A very small sample.** With 12 rows there are only so many distinct resamples, and the tails are guesswork.

Principle: what it does and does not do

Resampling estimates how much your answer moves from sample to sample. It knows nothing about whether the sample was the right one to take.

Two things a forest gives you free

- **Out-of-bag error.** Each tree is fitted on its own bootstrap resample, so about a third of the rows are left out of each tree. Score every row using only the trees that did not see it and you have a held-out estimate without holding anything out. Here OOB says $R^2 = 0.781$ and a real test set says 0.771.
- **A partial dependence curve.** Sweep one column across its range, holding the others as they are, and plot what the forest predicts. It gives a shape rather than a coefficient, and it is the closest a forest comes to answering "how much".

Saying it out loud

The same finding, three ways. Only the last is usable.

Too raw	"Permutation importance for Mumbai was 0.699 with a bootstrap CI of $[0.643, 0.745]$."
Too loose	"The model says location is the big one."
Right	"Of everything we recorded, whether the flat is in Mumbai is what the model leans on hardest, and resampling 40 times never changed that. Size is second. The random id we added as a check came out at zero, which is how we know the ranking is worth reading. None of this says how much rent goes up, only what the model uses."

In practice: the four parts

What it leaned on, how steady that was, what the noise column did, and the line saying this is not an effect size.

What to do when the model has no formula

1. **Want to know which columns the model used?** Permutation importance, not the default.
2. **Add a column of noise** and check it lands at zero.
3. **Want an interval on anything at all?** Resample with replacement, recompute, take percentiles.
4. **Resample whatever the quantity was computed from**, and leave any train/test split where it is.
5. **Want an honest error for a forest?** Out-of-bag, or a test set.
6. **Report the estimate from the data** and the interval from the resamples.
7. **Say what the interval does not cover:** bias, dependence, and whether the sample was the right one.

The traps

Trap: the ones that bite most often

- Reading an importance ranking as a ranking of effect sizes.
- Reporting the default importance on data with an id, a timestamp, or any column with many distinct values.
- Reading an importance score as having units or a direction.
- Resampling without replacement, which produces no spread at all.
- Resampling before a train/test split, so the same row lands on both sides.
- Quoting a bootstrap interval as though it covered sampling bias.
- Averaging the resamples and reporting that as the estimate.

What you should be able to do with software

You should be able to	What you would reach for
Fit a forest and get its held-out error	`RandomForestRegressor(oob_score=True)`
Get the default importance	`forest.feature_importances_`
Get permutation importance	`permutation_importance(forest, X_test, y_test)`
Draw one resample	`df.sample(len(df), replace=True)`
Turn resamples into an interval	`np.percentile(boot, [2.5, 97.5])`
Check an importance method	add a random column and refit

Where we go next

- Every interval so far has come from one of two places: a formula that assumed a distribution, or resampling that assumed almost nothing.
- **Unit 7** starts on where the formulas come from: probability, and what chance alone produces.
- The bootstrap returns there with a reason attached, rather than as a recipe that happens to work.